

Low Level Design (LLD)

IntelliPlant AI

AI-Powered Industrial Knowledge Intelligence Platform

Version: 1.0

Document Type: Low Level Design (LLD)

Prepared For: ET AI Hackathon 2026

Prepared By: Team IntelliPlant AI

Table of Contents

1. Introduction
 2. Folder Structure
 3. Microservices Design
 4. Database Design
 5. API Design
 6. AI Services
 7. LangGraph Workflow
 8. Agent Design
 9. Class Diagram
 10. Sequence Diagrams
 11. Worker Architecture
 12. Cache Layer
 13. Event Flow
 14. Error Handling
 15. Security Layer
-

1. Introduction

The Low-Level Design (LLD) defines the internal implementation details, class structures, services, modules, APIs, and workflows required for IntelliPlant AI.

2. Project Structure

```
intelliplant-ai/
```

```
backend/
|
├── src/
|
├── api/
|   ├── auth/
|   ├── document/
|   ├── search/
|   ├── chat/
|   ├── compliance/
|   └── rca/
|
├── agents/
|   ├── supervisor/
|   ├── expert/
|   ├── rca/
|   ├── compliance/
|   └── lessons/
|
├── services/
|   ├── ingestion/
|   ├── chunking/
|   ├── embedding/
|   ├── graph/
|   ├── rag/
|   ├── ocr/
|   └── retrieval/
|
├── db/
|   ├── postgres/
|   ├── neo4j/
|   └── qdrant/
|
├── workers/
|
├── models/
|
├── schemas/
|
├── middleware/
|
├── utils/
|
└── tests/
```

```
frontend/  
  
app/  
  
components/  
  
chat/  
  
dashboard/  
  
graph/  
  
documents/  
  
admin/
```

3. Microservices

Auth Service

Responsibilities

- Login
- JWT
- RBAC

Endpoints

```
POST /login  
POST /register  
POST /logout
```

Document Service

Responsibilities

- Upload files
- Metadata extraction
- File storage

Endpoints

```
POST /upload
GET /documents
DELETE /document/{id}
```

Search Service

Responsibilities

- Hybrid search
- Context fusion

Endpoints

```
POST /search
POST /chat
```

Agent Service

Responsibilities

- Multi-agent orchestration

Endpoints

```
POST /agent/query
```

4. Database Layer

PostgreSQL

Tables

users

```
id
name
email
```

```
password
role
created_at
```

documents

```
id
name
type
path
status
uploaded_at
```

chunks

```
id
document_id
chunk_text
embedding_id
```

audit_logs

```
id
user_id
action
timestamp
```

Neo4j Schema

Nodes

```
Equipment
Incident
Manual
Operator
Inspection
```

Regulation
Failure
WorkOrder

Relationships

CONNECTED_TO

FAILED_IN

CAUSES

REQUIRES

SIMILAR_TO

REFERENCES

Qdrant Collection

document_chunks

Fields

id

vector

metadata

document_id

chunk_text

5. API Design

Upload API

Request

```
{
  "file": "manual.pdf"
}
```

Response

```
{
  "status": "uploaded"
}
```

Search API

Request

```
{
  "query": "Why Pump P101 failed?"
}
```

Response

```
{
  "answer": "",
  "citations": [],
  "confidence": 0.91
}
```

6. Pydantic Models

UserSchema

```
class UserSchema(BaseModel):
    name: str
    email: str
    password: str
```

DocumentSchema

```
class DocumentSchema(BaseModel):  
    id:int  
    name:str  
    path:str
```

ChatSchema

```
class ChatSchema(BaseModel):  
    query:str
```

7. Document Pipeline

```
Upload  
↓  
OCR  
↓  
Metadata Extraction  
↓  
Entity Extraction  
↓  
Chunking  
↓  
Embedding  
↓  
Qdrant  
↓  
Graph Builder  
↓  
Neo4j
```

8. OCR Service

Technology

Docling

PaddleOCR

Methods

```
extract_text()

extract_tables()

extract_images()
```

Output

```
{
  "text": "",
  "tables": []
}
```

9. Chunking Service

Methods

```
semantic_chunk()

recursive_chunk()
```

Chunk Size

1024

Overlap

200

10. Embedding Service

Model

BGE-Large

Function

```
generate_embeddings()
```

Output

768 dimensions

11. Retrieval Engine

Components

Vector Search

Qdrant

BM25 Search

Keyword retrieval

Graph Search

Neo4j traversal

Context Fusion

Merge retrieved results

12. LangGraph Workflow

User Query

↓

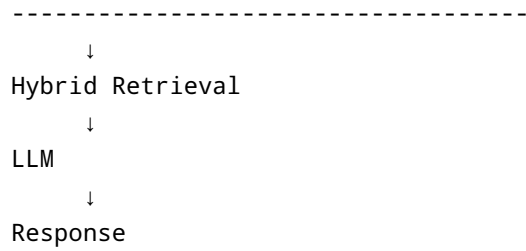
Supervisor Agent

↓

Intent Detection

↓

Expert Agent
RCA Agent
Compliance Agent
Lessons Agent



13. Agent Design

Supervisor Agent

Functions

```
route_query()  
  
manage_context()
```

Expert Agent

Functions

```
answer_question()  
  
retrieve_documents()
```

RCA Agent

Functions

```
find_similar_failures()

root_cause_analysis()
```

Compliance Agent

Functions

```
detect_gaps()

generate_report()
```

Lessons Agent

Functions

```
analyze_patterns()

recommend_actions()
```

14. Class Diagram

```
User
|
| 1
|
*----- Documents
|
*----- ChatHistory

Document
|
*----- Chunks

Chunk
|
```

*----- Embedding

Equipment

|

*----- Incident

Incident

|

*----- Failure

15. Sequence Diagram

Chat Flow

User

↓

Frontend

↓

API

↓

Supervisor Agent

↓

Retrieval Engine

↓

Qdrant + Neo4j

↓

Gemini/OpenAI

↓

Response

16. Worker Architecture

Technology

Celery + Redis

Workers

OCR Worker

Processes OCR

Embedding Worker

Creates embeddings

Graph Worker

Updates Neo4j

Notification Worker

Background alerts

17. Cache Layer

Technology

Redis

Stores

Session State

Recent Queries

Agent Memory

Frequently Asked Responses

TTL

30 minutes

18. Event Flow

Upload Event

Document Uploaded

↓

OCR Event

↓

Chunk Event

↓

Embedding Event

↓

Graph Event

↓

Index Event

19. Error Handling

OCR Failure

Retry

Embedding Failure

Queue processing

Neo4j Failure

Fallback mode

LLM Failure

Cached response

API Failure

Retry policy

20. Authentication Flow

User Login

↓

JWT Token

↓

Middleware

↓

Role Validation

↓

Access Granted

21. Roles

Admin

Full access

Manager

Analytics + Chat

Engineer

Upload + Search

Technician

Read-only

22. Logging

Store

User Logs

API Logs

Agent Logs

LLM Logs

Error Logs

23. Monitoring

Prometheus

Grafana

Metrics

Latency

Token Usage

Query Count

Failures

Agent Performance

24. File Storage

MinIO

Structure

documents/

manuals/

reports/

images/

inspection/

workorders/

25. Configuration

Environment Variables

POSTGRES_URL=

QDRANT_URL=

NEO4J_URI=

REDIS_URL=

GEMINI_API_KEY=

OPENAI_API_KEY=

MINIO_ENDPOINT=

26. Testing Layer

Unit Tests

Pytest

API Tests

FastAPI TestClient

Integration Tests

Database + Search

Agent Tests

LangSmith

27. Future Enhancements

SCADA Integration

IoT Sensors

Digital Twin

Predictive Maintenance

Voice Copilot

Mobile App

SAP Integration

Conclusion

This Low-Level Design document defines the internal implementation details of IntelliPlant AI. It serves as the blueprint for developers and AI engineers to implement the complete GraphRAG-based Industrial Knowledge Intelligence platform.